

Système expert : application à la résolution des tours de Hanoï.

1. Contexte

Nous souhaitons mettre en place un **système à base de règles** (système expert) qui, par inférence en chaînage, doit résoudre le problème des **tours de Hanoï**.

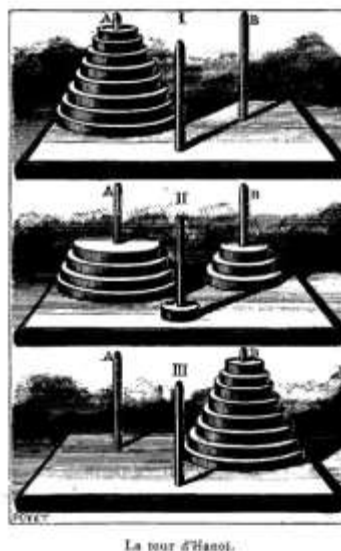
L'énoncé du problème des tours de Hanoï est le suivant (source Wikipedia) :

Les tours de Hanoï (ou tour d'Hanoï) sont un jeu de réflexion imaginé par le mathématicien français Édouard Lucas.

Le but est de déplacer une pile de disques de diamètres différents d'une tour de départ vers une tour d'arrivée, en utilisant une tour intermédiaire, et ceci en un **minimum de coups**, selon les règles :

1. On ne peut déplacer qu'un seul disque à la fois.
2. On ne peut poser un disque que sur un disque plus grand, ou sur un emplacement vide.

Ci-dessous, vous avez une illustration « historique » du jeu (source Wikipedia).



Nous considérons dans ce projet le cas **classique avec 3 tours et 3 disques** (de taille 1, 2 et 3). Nous référencerons les disques avec leur taille.

Nous proposons une classe Python pour représenter une configuration de jeu :

```
class Jeu_Hanoi:
```

```
    def __init__(self):
```

```
        self.pic = np.zeros([3,3],dtype= int)
```

```
        self.nombre_palet = np.zeros(3,dtype=int)
```

```
        pass
```

- `pic` : tableau NumPy 3x3 représentant les tours (indices 0 à 2). Exemple : `pic[b,0]` est la base de la tour `b`.

- nombre_palet : tableau contenant le nombre de disques présents sur chaque tour.

Situation initiale : tous les disques sont sur le premier pic rangés suivant une taille décroissante donc 3 -> 2 -> 1) :

```
jeu = Jeu_Hanoi()
```

```
jeu.nombre_palet[0] = 3
```

```
jeu.pic[0,0] = 3
```

```
jeu.pic[0,1] = 2
```

```
jeu.pic[0,2] = 1
```

Le reste est à 0 du fait de l'initialisation.

Situation finale désirée : tous les disques sont sur le dernier pic rangés suivant une taille décroissante 3 -> 2 -> 1 :

```
Jeu_final = Jeu_Hanoi()
```

```
Jeu_final.nombre_palet[2] = 3
```

```
Jeu_final.pic[2,0] = 3
```

```
Jeu_final.pic[2,1] = 2
```

```
Jeu_final.pic[2,2] = 1
```

2. Base experte

Ce problème peut se résoudre d'une manière optimale. Cependant nous allons mettre en place un système expert qui cherche une résolution à partir de règles simples.

Après échange avec un expert, nous avons identifié les conditions suivantes :

- On ne peut déplacer un disque que si le pic d'origine contient au moins un disque ;

Suggestion : cette condition peut être validée par exemple pour le pic b à partir de la valeur $jeu.nombre_palet[b] > 0$.

- On peut poser le disque supérieur d'un pic b sur un pic d uniquement si ce dernier est vide ou si son disque supérieur est plus grand.

Suggestion : cette condition peut être validée à partir de la comparaison de la valeur de $jeu.pic[b, jeu.nombre_palet[b]-1]$ et la valeur de $jeu.pic[d, jeu.nombre_palet[d]-1]$.

- Pour éviter les boucles, on interdit les déplacements menant à une situation déjà observée.

Suggestion : On associe à chaque situation un **nombre unique** (codage binaire) permettant de l'identifier et de l'enregistrer dans une liste. Par exemple à l'initialisation, le nombre peut se calculer suivant :

```
jeu.pic = [ [ 3 2 1 ] [ 0 0 0 ] [ 0 0 0 ] ] -> nombre en base 2 = 000 000 111
```

Si la situation est la suivante :

```
jeu.pic = [ [ 2 1 0 ] [ 3 0 0 ] [ 0 0 0 ] ] -> nombre en base 2 = 000 100 011
```

A la situation finale désirée :

Jeu.pic = [[0 0 0] [0 0 0] [3 2 1]] -> nombre en base 2 = 111 000 000

Ainsi à chaque étape de la résolution, la situation créée devient un nouveau fait que vous pouvez ajouter dans une liste Python par exemple appelée `situation_etudiee` qui enregistrera toutes les situations de jeu interdites pour le futur.

Enfin pour finaliser la base experte, l'expert nous indique qu'il faut appliquer les règles suivantes :

- Règle 1 : déplacer du pic 0 vers pic 1 si possible.
- Règle 2 : déplacer du pic 1 vers pic 2 si possible.
- Règle 3 : déplacer du pic 2 vers pic 1 si possible.
- Règle 4 : déplacer du pic 1 vers pic 0 si possible.
- Métarègle : en cas de conflit, appliquer la règle au plus petit indice.

Par exemple, la règle 1 se traduit par : si (le pic 0 est non vide) et (le déplacement du disque supérieur du pic 0 sur le pic 1 est autorisé) et (la situation résultante de ce déplacement n'a pas été observée) alors réaliser (le déplacement du disque supérieur du pic 0 sur le pic 1).

3. Réalisation

Vous développerez un moteur d'inférence en chaînage avant avec les fonctions suivantes :

- `nombre_situation(jeu)` : convertit une situation en nombre unique. Retourne ce nombre.
- `pic_vide(indice_pic, jeu)` : vérifie si un pic est vide. Retourne un Boolean.
- `regle_jeu(pic1, pic2, jeu)` : vérifie si un déplacement est autorisé. Retourne un Boolean
- `effectue_deplacement(pic1, pic2, jeu)` : effectue le déplacement. Modifie la situation `jeu`
- `situation_non_vue(pic1, pic2, jeu, situation_etudiee)` : empêche les répétitions. Retourne un Boolean

À chaque itération, le moteur :

1. Sélectionne une règle éligible.

Par exemple, si $(\text{NOT}(\text{pic_vide}(\text{indice_pic1}, \text{jeu})))$ et $(\text{regle_jeu}(\text{indice_pic1}, \text{indice_pic2}, \text{jeu}))$ et $(\text{situation_non_vue}(\text{indice_pic1}, \text{indice_pic2}, \text{jeu}, \text{situation_etudiee}))$ alors règle éligible

2. Effectue le déplacement.
3. Met à jour la liste des situations.
4. Incrémente le compteur de coups.

Le processus s'arrête dès que la **situation finale** est atteinte.

Analyse attendue

- Tester votre programme et afficher les étapes.
- Comparer le nombre de coups réalisés avec le nombre **optimal**.
- Réfléchir à l'ajout d'autres règles pour optimiser la résolution.
- (Optionnel) Étendre à d'autres cas (4 ou 5 disques, ou 4 tours).